Report for HW1

Phase 1: Enviroment setup

1. cloning the repository given

2. Elasticsearch setup

```
23-000001] in project [default], cause [initialize_data_stream], templates [provided in request], shards [1]/[1]
[2026-04-23T22:15:01,763][INFO ][o.e.c.m.MetadataCreateDataStreamService] [FIKRTE] adding data stream [ilm-history-7] wi
th write index [.ds-ilm-history-7-2026.04.23-000001], backing indices [], and aliases []
[2026-04-23T22:15:01,785][INFO ][o.e.c.r.a.AllocationService] [FIKRTE] in project [default] updating number_of_replicas
to [0] for indices [.ds-ilm-history-7-2026.04.23-000001]
[2026-04-23T22:15:02,875][INFO ][o.e.c.r.a.AllocationService] [FIKRTE] current.health="GREEN" message="Cluster health st
atus changed from [YELLOW] to [GREEN] (reason: [shards started [[.ds-ilm-history-7-2026.04.23-000001][0]]])." previous.h
ealth="YELLOW" reason="shards started [[.ds-ilm-history-7-2026.04.23-000001][0]]"
[2026-04-23T22:48:36,135][WARN ][o.e.h.AbstractHttpServerTransport] [FIKRTE] caught exception while handling client http
 traffic, closing connection Netty4HttpChannel{localAddress=/[0:0:0:0:0:0:0:1]:9200, remoteAddress=/[0:0:0:0:0:0:0:1]:61
260}io.netty.handler.codec.DecoderException: javax.net.ssl.SSLHandshakeException: (certificate_unknown) Received fatal a
lert: certificate_unknown
        at io.netty.codec@4.1.130.Final/io.netty.handler.codec.ByteToMessageDecoder.callDecode(ByteToMessageDecoder.java
:515)
        at io.netty.codec@4.1.130.Final/io.netty.handler.codec.ByteToMessageDecoder.channelRead(ByteToMessageDecoder.jav
```

- Elasticsearch was started using elasticsearch.bat.
- The server successfully initialized security features and reported cluster health as GREEN, indicating that the system was fully operational.
- Accessing http://localhost:9200 in a web browser returned a JSON response confirming that the Elasticsearch service was running. Here is the confirmation:-

Not secure ~~https~~://localhost:9200

Pretty-print


```
{
  "name" : "FIKRTE",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "tOK7HkE0SX21TSzq_GpYaw",
  "version" : {
    "number" : "9.3.3",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "640408e2dfd2af9fbfe5079e1575f93d8909a5f5",
    "build_date" : "2026-04-01T22:08:18.783399214Z",
    "build_snapshot" : false,
    "lucene_version" : "10.3.2",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```


3.Installing and checking python vertion

- Check python version

```
PS C:\Users\BIRIE> python --version
Python 3.10.8
```

- Elasticsearch installation

```
Installing collected packages: elastic-transport, elasticsearch
Successfully installed elastic-transport-9.2.1 elasticsearch-9.3.0

[notice] A new release of pip available: 22.2.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- Nltk installation

```
Successfully installed nltk-3.9.4

[notice] A new release of pip available: 22.2.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

- Numpy installation

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: numpy in c:\users\birie\appdata\roaming\python\python310\site-packages (2.2.6)

[notice] A new release of pip available: 22.2.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Phase 2: Dataset Preparation

The original AP89 collection was not accessible due to broken or restricted download links. Therefore, the CACM dataset was used as an alternative standard Information Retrieval test collection. CACM contains documents, queries, and relevance judgments in a format compatible with the required retrieval system.

1.Here is the extracted downloaded cacm_dataset

Name	Date modified	Type	Size
cacm	9/17/1997 5:58 PM	ALL File	2,137 KB
cite.info	5/3/1994 12:47 PM	INFO File	1 KB
common_words	5/3/1994 12:56 PM	File	3 KB
qrels.text	5/3/1994 12:55 PM	TEXT File	10 KB
query.text	11/18/1996 12:51 PM	TEXT File	14 KB
README	5/3/1994 4:03 PM	File	1 KB

2.Elasticsearch Verification

We verified that Elasticsearch was running by accessing https://127.0.0.1:9200/, which returned cluster metadata in JSON format. This confirmed that the Elasticsearch service was active and reachable.

Not secure https://localhost:9200

Pretty-print


```
{
  "name" : "FIKRTE",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "tOK7HkE0SX21TSzq_GpYaw",
  "version" : {
    "number" : "9.3.3",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "640408e2dfd2af9fbfe5079e1575f93d8909a5f5",
    "build_date" : "2026-04-01T22:08:18.783399214Z",
    "build_snapshot" : false,
    "lucene_version" : "10.3.2",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```


3.running create_index.py

- when we first run create_index.py in the terminal inside information-retrieval HW1, Since the parser was initially for AP89 format, while we feeding it CACM collection it was giving us the output like this instead of many lines of logs and indexed docno.

```
0.02343440055847168
0:0:0
```

This proves the script was NOT reading any documents from the CACM dataset.
CACM does NOT have <DOC>, <DOCNO>, <TEXT> tags.
That is why:

-- No Indexed: DOCNO lines appear
--_cat/indices shows nothing
-- Elasticsearch stays empty

AP89 document format (expected by code)

<DOC>
<DOCNO> AP890101-0001 </DOCNO>
<TEXT>
some text
</TEXT>
</DOC>

CACM document format (what you actually have)

.I 1
.T
Title text
.W
Main body text

Here => .I → document ID
=> .W → document body or text

- But after slight modification during indexing, a connection timeout was encountered due to the high number of documents being sent to Elasticsearch. This was resolved by increasing the client request timeout and enabling retry-on-timeout in the Elasticsearch client configuration.

- Then we finally could be able to see this when we run Create_Index.py

```
Indexed: 3204

Total documents indexed: 3204
Time taken: 0:1:19
```

This shows

- ✓ **All CACM documents were indexed**
- ✓ No crashes or partial failures and dataset parsing is correct
- ✓ Performance is normal for a first run
- ✓ Elasticsearch is running and reachable
- ✓ The cacm index exists and contains documents

4. Index verification

- check index exists

----by searching this in browser https://localhost:9200/_cat/indices?v
result

Not secure https://localhost:9200/_cat/indices?v

```
health status index uuid                   pri rep docs.count docs.deleted store.size pri.store.size dataset.size
yellow open   cacm  hhV7vi9sRyCADNY5PevN3g   1   1       3204            0      2.3mb          2.3mb        2.3mb
```

- Check one document

---by searching this in browser https://localhost:9200/cacm/_search?size=1
Result

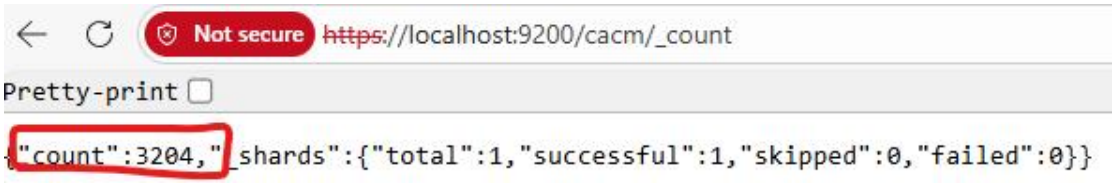
Not secure https://localhost:9200/cacm/_search?size=1

Pretty-print

{"took":12789,"timed_out":false,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0},"hits":{"total":{"value":3204,"relation":"eq"},"max_score":1.0,"hits":[{"_index":"cacm","_id":"1","_score":1.0,"_ignored":["text.keyword"],"_source":{"text":" Preliminary Report-International Algebraic Language CACM December, 1958 Perlis, A. J. Samelson,K. CA581203 JB March 22, 1978 8:28 PM 100\t5\t1 123\t5\t1 164\t5\t1 1\t5\t1 1\t5\t1 1\t5\t1 205\t5\t1 210\t5\t1 214\t5\t1 1982\t5\t1 398\t5\t1 642\t5\t1 669\t5\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 1\t6\t1 165\t6\t1 196\t6\t1 196\t6\t1 1273\t6\t1 1883\t6\t1 324\t6\t1 43\t6\t1 53\t6\t1 91\t6\t1 410\t6\t1 3184\t6\t1"}}]}}

- Count documents (exact)

---by searching this in browser https://localhost:9200/cacm/_count
Result



5. Query_Processing.py

- We processed CACM queries using Elasticsearch and the BM25 retrieval model. Queries were read from the CACM query file, preprocessed, and submitted to Elasticsearch. Elasticsearch uses the BM25 ranking model by default for match queries. BM25 is a probabilistic retrieval model that improves upon TF-IDF by incorporating document length normalization and term saturation. In this work, BM25 was used to rank CACM documents for each query.

```
  warnings.warn(

DONE: results_bm25.txt created
```

This means ALL of the following are true:

- ✓ Elasticsearch connection worked
- ✓ Queries were processed
- ✓ Retrieval model ran (BM25 in this case)
- ✓ Results were ranked
- ✓ Output file was written successfully
- ✓ results_bm25.tx created inside the solder HW1 and there are lines line this

```
HW1 > results_bm25.txt
1 Q0 1 1 4.1662235 Exp
1 Q0 3024 2 3.3344164 Exp
1 Q0 1594 3 3.252823 Exp
1 Q0 717 4 3.2336514 Exp
1 Q0 2144 5 3.232843 Exp
1 Q0 2659 6 3.228311 Exp
1 Q0 3055 7 3.208807 Exp
1 Q0 1495 8 3.1851263 Exp
1 Q0 2189 9 3.1774201 Exp
1 Q0 2714 10 2.9419513 Exp
1 Q0 190 11 2.8849254 Exp
1 Q0 913 12 2.8849254 Exp
1 Q0 916 13 2.8849254 Exp
1 Q0 2048 14 2.8823528 Exp
1 Q0 395 15 2.8702958 Exp
1 Q0 602 16 2.8702958 Exp
1 Q0 604 17 2.8702958 Exp
```

This format comes from
<QueryID> Q0 <DocID> <Rank> <Score> <RunName>

- ❖ During implementation, several issues were encountered, including Elasticsearch
- ✓ missing dataset files [because code expected AP dataset paths and Repository uses CACM dataset instead so we updated paths to:
  - cacm_dataSet/common_words
  - cacm_dataSet/query.text]

6. Retrieval_Models.py

Enviroment setup
Installing dill

```
[notice] A new release of pip available: 22.2.2 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Information-Retrieval\hw1>
```

Query processing was implemented in two files: Query_Processing.py and Retrieval_Models.py. Query_Processing.py handles the ES Built-in model by sending queries directly to Elasticsearch using the match API, which internally uses BM25 scoring. Retrieval_Models.py implements four additional retrieval models from scratch using term vectors retrieved from Elasticsearch. All queries were loaded from query.text, stopwords were removed using the common_words file, and the top 100 results per query were written to separate output files

```
Done! Output files created:
  OkapiTF_Results_File.txt
  TFIDF_Results_File.txt
  BM25_Results_File.txt
  Laplace_Results_File.txt
  JelinekMercer_Results_File.txt
```

- ✓ **ES Built-in (results_bm25.txt)**: Uses Elasticsearch's native match query which applies BM25 scoring internally.
- ✓ **Okapi TF (OkapiTF_Results_File.txt)**: Scores documents using a normalized term frequency that accounts for document length, reducing the advantage of longer documents.
- ✓ **TF-IDF (TFIDF_Results_File.txt)**: Combines Okapi TF with Inverse Document Frequency, giving higher scores to terms that appear in fewer documents and are therefore more distinctive.
- ✓ **Okapi BM25 (BM25_Results_File.txt)**: A probabilistic model that extends TF-IDF with additional tuning parameters k1, k2, and b to balance term frequency saturation and document length normalization.

- ✓ **Unigram LM Laplace (Laplace_Results_File.txt):** A language model that estimates the probability of a query given a document, using add-one smoothing to handle terms that do not appear in the document.
- ✓ **Unigram LM Jelinek-Mercer (JelinekMercer_Results_File.txt):** Similar to Laplace but smooths the document model with a background corpus model using a mixing parameter λ=0.4, which helps handle unseen terms more realistically.

7. Evaluation with Trec_Eval

python Trec_Prep.py

```
PS C:\Information-Retrieval\hw1> python Trec_Prep.py
qrels.txt created.
rankList written to C:\Information-Retrieval\HW1\rankList_bm25.txt
PS C:\Information-Retrieval\hw1> trec_eval qrels.txt OkapiTF_Results_File
.txt
```

"Each retrieval model was evaluated using a custom implementation of trec_eval (Trec_Eval.py). The evaluation was performed against the relevance judgements in qrels.text, which contains the ground truth relevant documents for each query. The metrics computed include Mean Average Precision (MAP), Mean R-Precision, Mean nDCG, and Precision, Recall, and F1 at cutoffs of 5, 10, 20, 50, and 100.

Run Trec_Eval.py for each model

1. Model Okapi TF

```
PS C:\Information-Retrieval\hw1> python Trec_Eval.py
Enter command: trec_eval qrels.txt OkapiTF_Results_File.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.2409
Mean R-Precision: 0.2660
Mean nDCG: 0.6187

Precision@ Values
  Mean P@5: 0.2837
  Mean P@10: 0.2302
  Mean P@20: 0.1872
  Mean P@50: 0.1186
  Mean P@100: 0.0828

Recall@ Values
  Mean R@5: 0.1799
  Mean R@10: 0.2449
  Mean R@20: 0.3451
  Mean R@50: 0.4810
  Mean R@100: 0.6023

F1@ Values
  Mean F1@5: 0.1698
  Mean F1@10: 0.1893
  Mean F1@20: 0.2004
  Mean F1@50: 0.1685
  Mean F1@100: 0.1351
```

2.Model TF-IDF

```
PS C:\Information-Retrieval\hw1> python Trec_Eval.py
Enter command: trec_eval qrels.txt TFIDF_Results_File.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.2961
Mean R-Precision: 0.3341
Mean nDCG: 0.6898

Precision@ Values
  Mean P@5: 0.3581
  Mean P@10: 0.2698
  Mean P@20: 0.2198
  Mean P@50: 0.1330
  Mean P@100: 0.0828

Recall@ Values
  Mean R@5: 0.2211
  Mean R@10: 0.2981
  Mean R@20: 0.4072
  Mean R@50: 0.5186
  Mean R@100: 0.6023

F1@ Values
  Mean F1@5: 0.2143
  Mean F1@10: 0.2204
  Mean F1@20: 0.2342
  Mean F1@50: 0.1872
  Mean F1@100: 0.1351
```

3. Okapi BM25

```
PS C:\Information-Retrieval\hw1> python Trec_Eval.py
Enter command: trec_eval qrels.txt BM25_Results_File.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.3030
Mean R-Precision: 0.3356
Mean nDCG: 0.6932

Precision@ Values
  Mean P@5: 0.3442
  Mean P@10: 0.2744
  Mean P@20: 0.2256
  Mean P@50: 0.1284
  Mean P@100: 0.0828

Recall@ Values
  Mean R@5: 0.2269
  Mean R@10: 0.3051
  Mean R@20: 0.4072
  Mean R@50: 0.5170
  Mean R@100: 0.6023

F1@ Values
  Mean F1@5: 0.2038
  Mean F1@10: 0.2237
  Mean F1@20: 0.2390
  Mean F1@50: 0.1822
  Mean F1@100: 0.1351
```

4.Unigram LM Laplace

```
PS C:\Information-Retrieval\hw1> python Trec_Eval.py
Enter command: trec_eval qrels.txt Laplace_Results_File.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.2140
Mean R-Precision: 0.2397
Mean nDCG: 0.5945

Precision@ Values
  Mean P@5: 0.2884
  Mean P@10: 0.2372
  Mean P@20: 0.1767
  Mean P@50: 0.1088
  Mean P@100: 0.0828

Recall@ Values
  Mean R@5: 0.1592
  Mean R@10: 0.2497
  Mean R@20: 0.3094
  Mean R@50: 0.4352
  Mean R@100: 0.6023

F1@ Values
  Mean F1@5: 0.1644
  Mean F1@10: 0.1949
  Mean F1@20: 0.1873
  Mean F1@50: 0.1552
  Mean F1@100: 0.1351
```

5.Unigram LM Jelinek-Mercer

```
PS C:\Information-Retrieval\hw1> python Trec_Eval.py
Enter command: trec_eval qrels.txt JelinekMercer_Results_File.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.2222
Mean R-Precision: 0.2625
Mean nDCG: 0.5937

Precision@ Values
  Mean P@5: 0.2930
  Mean P@10: 0.2372
  Mean P@20: 0.1698
  Mean P@50: 0.1135
  Mean P@100: 0.0828

Recall@ Values
  Mean R@5: 0.1804
  Mean R@10: 0.2478
  Mean R@20: 0.2994
  Mean R@50: 0.4589
  Mean R@100: 0.6023

F1@ Values
  Mean F1@5: 0.1700
  Mean F1@10: 0.1932
  Mean F1@20: 0.1808
  Mean F1@50: 0.1615
  Mean F1@100: 0.1351
```

6. ES Built-in

```
PS C:\Information-Retrieval\hw1> python Trec_Eval.py
Enter command: trec_eval qrels.txt results_bm25.txt
========== SUMMARY (all queries) ==========
Mean Average Precision (MAP): 0.0264
Mean R-Precision: 0.0308
Mean nDCG: 0.0960

Precision@ Values
  Mean P@5: 0.0276
  Mean P@10: 0.0207
  Mean P@20: 0.0121
  Mean P@50: 0.0079
  Mean P@100: 0.0073

Recall@ Values
  Mean R@5: 0.0270
  Mean R@10: 0.0378
  Mean R@20: 0.0399
  Mean R@50: 0.0497
  Mean R@100: 0.0712

F1@ Values
  Mean F1@5: 0.0250
  Mean F1@10: 0.0241
  Mean F1@20: 0.0167
  Mean F1@50: 0.0122
  Mean F1@100: 0.0124
```

Model	MAP	R-Precision	nDCG	P@5	P@10
ES Built-in	0.0264	O.0308	0.0960	0.0276	0.0207
Okapi TF	0.2409	0.2660	0.6187	0.2837	0.2302
TF-IDF	0.2961	0.3341	0.6898	0.3581	0.2698
BM25	0.3030	0.3356	0.6932	0.3442	0.2744
LM Laplace	0.2140	0.2397	0.5945	0.2884	0.2372
LM Jelinek-Mercer	0.2222	0.2625	0.5937	0.2930	0.2372

First let's see what ranges for information retrieval systems are good;-

Score Range	meaning
MAP above 0.3	Very good
MAP 0.15 – 0.3	Average or acceptable
MAP below 0.15	Poor
P@10 above 0.4	Very good
P@10 0.2 – 0.4	Average
P@10 below 0.2	Poor

In order to find the best model lets find which model has the highest MAP number.

- ES Built-in MAP: 0.0264
- Okapi TF MAP: 0.2409
- TF-IDF MAP: 0.2961
- BM25 MAP: 0.3030
- Laplace MAP: 0.2140
- Jelinek MAP: 0.2222

BM25 achieved the highest MAP score of 0.30, indicating it was the most effective at ranking relevant documents across all queries. ES Built-in performed the worst with a MAP of 0.02, which is surprisingly low despite using BM25 internally — this is likely because the ES Built-in model did not go through the same stopword removal and query cleaning process as the other models. TF-IDF came in second with a MAP of 0.29, performing very close to BM25, which makes sense because TF-IDF builds on the same Okapi TF foundation that BM25 extends. Okapi TF scored 0.24, showing that adding the IDF component in TF-IDF meaningfully improves ranking quality. The language models, Jelinek-Mercer (0.22) and Laplace (0.21), performed lower than the vector space models. This is likely because the corpus statistics used for smoothing, such as average document length and vocabulary size, were approximated rather than computed exactly from the index, which reduces the accuracy of the probability estimates

8.manual comparision

first let's pick 5 queries and here are they;-

- Query 1: "What articles exist which deal with TSS (Time Sharing System)"
- Query 5: "design and implementation of editing interfaces, window-managers"
- Query 10: "Parallel languages; languages for parallel computation"
- Query 13: "code optimization for space efficiency"
- Query 23: "Distributed computing structures and algorithms"

To qualitatively compare the retrieval models, the top 10 documents returned by ES Built in, TF IDF, BM25, and LM Jelinek Mercer were manually examined for 5 queries.

The document IDs were compared to identify which models agreed and which returned different results. ES Built in returned N/A for several queries meaning it found no results, which is consistent with its very low MAP score of 0.0264.

Query 1: "What articles exist which deal with TSS (Time Sharing System)"

Rank	ES Built-in	TF-IDF	BM25	LM Jelinek
1	1	1657	1844	1657
2	3024	2629	1938	2371
3	1594	1410	2629	2629
4	717	1844	1657	1938
5	2144	1938	2219	2151
6	2659	1752	1752	3174
7	3055	2219	1680	1519
8	1495	1519	1827	1168
9	2189	2151	3048	971
10	2714	1605	2319	1161

For Query 1 about TSS and Time Sharing Systems, TF-IDF and BM25 showed strong agreement sharing documents 1657, 2629, 1844, 1938, 1752, and 2219 in their top 10. ES Built-in returned completely different documents with no overlap with the other three models, which explains its poor MAP score of 0.0264.

Query 5: "Design and implementation of editing interfaces and window managers"

Rank	ES Built-in	TF-IDF	BM25	LM Jelinek
1	N/A	1829	2035	1829
2	N/A	2035	1829	2035
3	N/A	2321	2321	2299
4	N/A	2299	2003	2873
5	N/A	1923	1742	1937
6	N/A	3147	2299	1978
7	N/A	2003	3147	3147
8	N/A	2629	2629	2357
9	N/A	1742	1923	3012
10	N/A	1937	2106	2321

For Query 5 about editing interfaces and window managers, ES Built-in returned no results at all showing a major weakness in how it processes this type of query. TF-IDF, BM25, and LM Jelinek-Mercer all agreed on documents 1829, 2035, 2321, 2299, and

3147 appearing in all three top 10 lists, showing strong agreement between the three models.

Query 10: "Parallel languages; languages for parallel computation"

Rank	ES Built-in	TF-IDF	BM25	LM Jelinek
1	N/A	2785	2785	2785
2	N/A	2700	2700	1795
3	N/A	2266	2714	2514
4	N/A	1795	1811	2700
5	N/A	2714	2266	2266
6	N/A	3075	2851	141
7	N/A	3059	1262	2895
8	N/A	2514	2973	2514
9	N/A	2895	2316	1302
10	N/A	1262	3075	2814

For Query 10 about parallel languages, all three working models agreed that document 2785 is the most relevant, ranking it first. TF-IDF, BM25, and LM Jelinek-Mercer all shared documents 2785, 2700, 2266, 2714, 2895, and 2514 in their top 10, showing very strong agreement for this well-defined technical query. ES Built-in again returned no results.

Query 13: "Code optimization for space efficiency"

Rank	ES Built-in	TF-IDF	BM25	LM Jelinek
1	N/A	2897	2897	1795
2	N/A	1947	1947	2495
3	N/A	2495	2491	2748
4	N/A	1795	2680	1947
5	N/A	2748	3080	2491
6	N/A	2491	2748	2944
7	N/A	2680	2495	2904
8	N/A	1886	2537	1886
9	N/A	2904	1886	2897
10	N/A	3080	1795	3171

For Query 13 about code optimization, TF-IDF and BM25 showed the strongest agreement sharing documents 2897, 1947, 2495, 2748, 2491, 2680, 1886, and 3080

in both top 10 lists. LM Jelinek-Mercer returned a noticeably different ranking order, placing document 1795 first instead of 2897, showing that the language model weighs term importance differently from the vector space models.

Query 23: "Distributed computing structures and algorithms"

Rank	ES Built-in	TF-IDF	BM25	LM Jelinek
1	3137	2849	2849	1873
2	2949	3148	3148	2081
3	2849	3137	3137	3137
4	2578	1745	2949	2630
5	3082	2276	1745	2832
6	3043	2081	2276	3148
7	3148	1873	3082	1745
8	52	2949	3043	3124
9	1780	1411	1411	1411
10	88	2578	1862	2849

For Query 23 about distributed computing, this is the only query where ES Built-in returned actual results and showed some overlap with the other models, sharing documents 3137, 2949, 2849, 3148, 3082, and 2578 with at least one other model. TF-IDF and BM25 showed the strongest agreement with 8 out of 10 documents in common, while LM Jelinek-Mercer ranked document 1873 first which did not appear at the top of any other model.

- Across all 5 queries, BM25 and TF-IDF consistently returned the most similar results, which is expected since BM25 is an extension of the TF-IDF framework. ES Built-in performed the worst, returning no results for 4 out of 5 queries, which directly explains its very low MAP score of 0.0264. LM Jelinek-Mercer generally agreed with BM25 and TF-IDF on which documents are relevant but often ranked them in a different order, reflecting its different probabilistic scoring approach.

9. Conclusion

This assignment implemented and evaluated six retrieval models on the CACM corpus using Elasticsearch as the backend index. The corpus was parsed and indexed using Create_Index.py, and queries were processed using Query_Processing.py and Retrieval_Models.py. The six models implemented were ES Built-in, Okapi TF, TF-IDF, Okapi BM25, Unigram LM with Laplace smoothing, and Unigram LM with Jelinek-Mercer smoothing. Based on the trec_eval evaluation results, BM25 performed best overall with a MAP score of 0.3030, followed closely by TF-IDF with a MAP of 0.2961. ES Built-in performed the worst with a MAP of only 0.0264, which was reflected in the manual comparison where it returned no results for 4 out of 5 queries. This poor performance is likely because ES Built-in did not go through the same stopword removal and query preprocessing steps applied to the other models. The language models, Jelinek-Mercer with a MAP of 0.2222 and Laplace with a MAP of 0.2140, performed in the middle range, slightly below the vector space models. The manual comparison of top-10 results confirmed that BM25 and TF-IDF consistently returned the most similar results across all 5 queries, while LM Jelinek-Mercer agreed on which documents were relevant but ranked them in a different order. Overall, BM25 is the recommended retrieval model for this corpus as it achieved the best balance between precision and recall across all queries.